ACADEMIC ENGINEERING SPECIFICATION • AWT & SOFTWARE ENGINEERING

HaritAyan (हरित अयन)

Technical Module Working Architecture & End-to-End Data Flow Pipeline

Comprehensive Mapping of Data Ingestion Sources, MongoDB Atlas Document Schemas, REST Controllers & Frontend UI Presentation

Geographic Scope: Vadodara District, Gujarat, India
AI Engine: Dual-Engine (Edge TF.js + Cloud Gemini 1.5 Pro)
Runtime: React 18 + Node.js / Express + MongoDB Atlas
Coursework: Advanced Web Technologies (AWT / SE)

1. Executive Architecture & Closed-Loop Design

HaritAyan is an enterprise-grade smart agriculture and urban heat mitigation platform architected specifically for the microclimates, cropping patterns, and APMC agricultural markets of **Vadodara District, Gujarat**. The platform bridges foliar computer vision disease diagnostics, urban heat island (UHI) thermodynamics, live commodity price intelligence, precision root-zone irrigation timing, and smallholder farm equipment subsidies into a single closed-loop ecosystem:

Data Ingestion (Sensors/APIs/Vision) → Backend REST Controllers → MongoDB Atlas Cloud Schemas → React 18 High-Contrast UI → Bilingual English/Gujarati Presentation

2. Master Data Topology Matrix

Module Name	Where Data Comes From	Where Data is Stored	Processing Backend Files	Frontend UI Display
1. Plant Disease Vision Scanner	User leaf photo (webcam/file), client TF.js MobileNetV2 edge model, Google Gemini 1.5 Pro API	MongoDB Atlas `scans` collection (Scan model) + `localStorage` offline cache	`routes/scanRoutes.js` `controllers/scanController.js` `services/geminiService.js`	`pages/PlantScanner.jsx` Confidence gauge, symptom card, organic remedies in EN & GU
2. Urban Heat & Canopy Simulator	Landsat/MODIS thermal infrared satellite data, Open-Meteo ambient temp, Vadodara ward polygons	Backend in-memory thermal index, `heatController.js` ward grid, client simulation state	`routes/heatRoutes.js` `controllers/heatController.js`	`pages/HeatMap.jsx` `UrbanHeat2DMap.jsx` (Leaflet) `TreeCanopy3D.jsx` (Three.js)
3. Real-Time APMC Mandi Engine	Data.gov.in / Agmarknet API, Sayajigunj, Padra, Karjan, Dabhoi, Savli trade receipts, admin overrides	MongoDB Atlas `mandiprices` collection (MandiPrice model) seeded via `seedMandiData.js`	`routes/mandiRoutes.js` `controllers/mandiController.js` `utils/mandiData.js`	`pages/MandiPrices.jsx` Live price ticker, 24h trend delta, commodity bar charts, EN/GU
4. Smart Irrigation & Power Timer	Crop Kc coefficients (Cotton, Banana, Tomato), Goradu/Black soil moisture deficit, MGVCL 3-phase power roster	MongoDB Atlas `irrigationlogs` collection (IrrigationLog model) + client pump session storage	`routes/irrigationRoutes.js` `models/IrrigationLog.js`	`pages/IrrigationTimer.jsx` Live countdown timer, audio chime, water saved counter, power shift toggle
5. Agro-Weather & IMD Alerts	Open-Meteo API (22.3072°N, 73.1812°E), IMD Ahmedabad radar alerts, admin emergency broadcast	MongoDB Atlas `weatheralerts` collection (WeatherAlert model)	`routes/weatherRoutes.js` `models/WeatherAlert.js`	`pages/Weather.jsx` `components/AlertCard.jsx` 7-day hourly forecast, heat index alert
6. Krishi Tools Marketplace	Vadodara 7-Taluka Custom Hiring Centers (CHCs), Gujarat i-Khedut 40–60% state subsidy slab rules	MongoDB Atlas `equipments` & `equipmentbookings` collections	`routes/equipmentRoutes.js` `models/Equipment.js` `models/EquipmentBooking.js`	`pages/Marketplace.jsx` Equipment cards, booking modal, live i-Khedut subsidy calculator
7. District Admin Operations Hub	MongoDB Atlas shard ping latency, Node.js process.memoryUsage(), dual-engine AI token consumption ledger	In-memory live system telemetry, MongoDB Atlas collections, ledger metrics	`routes/adminRoutes.js` `server.js`	`pages/AdminPanel.jsx` Atlas ping chart, memory telemetry, AI cost ledger, alert dispatcher
8. Multi-Role Auth & User Desks	Signup page 3-way role selector, login forms, 1-click live demo switcher, farmer query desk	MongoDB Atlas `users` & `queries` collections (User & Query models, bcrypt hashed)	`routes/authRoutes.js` `controllers/authController.js` `routes/queryRoutes.js`	`pages/LoginPage.jsx` `components/AuthModal.jsx` `pages/UserProfile.jsx` (Farmer/Officer)
9. HaritAyan Sahayak Chatbot	Regional agronomic decision tree rules, Gujarati botanical dictionary, platform routing table	Client in-memory conversational dialogue state & local session storage	Offline rule engine embedded in frontend client bundle	`components/Chatbot.jsx` Floating assistant widget, multilingual NLP query matcher, quick chips

Module 1: Dual-Engine Foliar Plant Pathology & Disease Scanner

AI Computer Vision & Edge Telemetry

Diagnoses visible fungal spots, bacterial blights, viral curls, and pest attacks on staple Gujarat cash crops (Cotton, Castor, Pigeon Pea, Tomato, Okra). Employs an ultra-fast in-browser **TensorFlow.js edge model** for zero-latency offline diagnostics with seamless failover to **Google Gemini 1.5 Pro Multimodal Cloud Vision** for deep agronomic reasoning.

📥 WHERE DATA COMES FROM

- **User Upload / Camera:** Leaf image captured via device camera or file input (JPEG/PNG up to 10MB).
- **Crop Selection:** Farmer specifies target crop (Cotton, Castor, Pigeon Pea, Tomato, Okra).
- **Edge Inference:** In-browser MobileNetV2 extracts foliar feature tensors directly on client GPU/CPU.
- **Cloud Multimodal Vision:** Base64 leaf image payload dispatched to Google Gemini 1.5 Pro API with tailored prompt.

💾 WHERE DATA IS STORED

- **MongoDB Atlas Collection:** `scans`
- **Model File:** `backend/src/models/Scan.js`
- **Stored Schema Fields:** `userId`, `crop`, `disease`, `confidence`, `severity`, `symptoms` (EN/GU), `organicRemedies` (EN/GU), `chemicalTreatments`, `preventionTips`, `imageUrl`, `diagnosedAt`, `inferenceEngine`.
- **Client Fallback:** Scans synced to `localStorage('haritayan_scan_history')` when offline.

⚙️ BACKEND PROCESSING FILES

- `backend/src/routes/scanRoutes.js`: Maps `POST /api/scan` (Multer upload) & `GET /api/scan/history`.
- `backend/src/controllers/scanController.js`: Handles image buffering, invokes Gemini API or rule-based fallback, and persists record to MongoDB.
- `backend/src/services/geminiService.js`: System instructions requesting JSON schema with bilingual symptoms and biological treatments (Neem oil, Trichoderma).
- `backend/src/middleware/uploadMiddleware.js`: Validates MIME types & stores leaf photos in `public/uploads/`.

🖥️ FRONTEND UI PRESENTATION

- `frontend/src/pages/PlantScanner.jsx`: Drag-and-drop zone, camera viewfinder, engine toggle (Edge TF.js vs Cloud Gemini), circular confidence gauge.
- `frontend/src/services/scanService.js`: Dispatches FormData to backend REST API.
- `frontend/src/services/tfjsModelService.js`: Initializes TensorFlow.js WebGL backend, tensor normalizations, and MobileNet inference.
- `frontend/src/components/TensorFlowLabModal.jsx`: Benchmarks client hardware inference latency (ms) & GPU memory.

END-TO-END DATA FLOW PIPELINE

1. Farmer snaps photo of diseased cotton leaf on `PlantScanner.jsx`.
2. Image is pre-processed on canvas. If offline, `tfjsModelService.js` classifies locally; if online, `scanService.js` sends multipart POST to `/api/scan`.
3. `uploadMiddleware.js` saves image to disk; `scanController.js` prompts Gemini 1.5 Pro with image buffer.
4. Gemini returns JSON diagnostics; `Scan.create()` persists result in MongoDB Atlas `scans` collection.
5. Frontend renders confidence badge, disease name in Gujarati/English, severity meter, and step-by-step bio-fungicide remedies.

Module 2: Urban Heat Island (UHI) Observatory & 3D Canopy Simulator

Thermal Microclimate & 3D Simulation

Models thermodynamic surface temperatures across 8 major municipal wards of Vadodara (Sayajigunj, Alkapuri, Makarpura, Manjalpur, Karelibaug, Gotri, Gorwa, Waghodia Road). Computes microclimatic cooling yields (ΔT up to -3.4°C) derived from simulated canopy expansion using native Gujarat shade trees (Neem, Peepal, Banyan, Jamun).

WHERE DATA COMES FROM

- **Thermal Satellite Baselines:** Empirical Landsat-8/9 Thermal Infrared Sensor (TIRS) surface temperature baselines for Vadodara municipal wards.
- **Live Weather Feed:** Open-Meteo API live ambient temperature, humidity, and solar irradiation at Vadodara coordinates.
- **Botanical Transpiration Factors:** Evapotranspiration cooling index $\Eeta$ and canopy leaf area index (LAI) for native trees.
- **User Parameters:** Tree count slider (0 - 50,000) and species selection.

WHERE DATA IS STORED

- **Backend Ward Registry:** `backend/src/controllers/heatController.js` contains calibrated ward coordinates, baseline temperatures, vegetation index (NDVI), and population densities.
- **Database Schema:** Cached microclimatic observations stored in MongoDB Atlas `systemmetrics`.
- **Client State:** Current ward selection, simulated tree density, and dynamic temperature deltas maintained in React component state.

BACKEND PROCESSING FILES

- `backend/src/routes/heatRoutes.js`: Exposes `GET /api/heat/wards` & `POST /api/heat/simulate`.
- `backend/src/controllers/heatController.js`: Implements cooling equation: `ΔT = -α · ln(1 + treeCount / 5000) · speciesFactor`.
- Calculates ward-specific heat vulnerability score (Low, Moderate, High, Extreme).

FRONTEND UI PRESENTATION

- `frontend/src/pages/HeatMap.jsx`: Ward selector, tree planting interactive slider, dynamic thermal gauge, cooling delta badge.
- `frontend/src/components/UrbanHeat2DMap.jsx`: Leaflet map with CartoDB Voyager tiles rendering color-coded ward heat polygons.
- `frontend/src/components/TreeCanopy3D.jsx`: Three.js WebGL canvas rendering 3D tree canopies, animated solar rays, and cooling thermal plumes.

END-TO-END DATA FLOW PIPELINE

1. User selects "Makarpura Industrial Zone" (Baseline 41.2°C) on `HeatMap.jsx`.
2. User adjusts slider to simulate planting 12,500 Neem trees.
3. `heatController.js` calculates evapotranspiration cooling: $\Delta T = -2.4^\circ\text{C}$, reducing surface temp to 38.8°C.
4. `UrbanHeat2DMap.jsx` updates ward polygon color from red to orange-amber; `TreeCanopy3D.jsx` visualizes lush 3D canopy coverage.

Module 3: Real-Time APMC Mandi Market Price Engine

Agricultural Market Price Intelligence

Tracks daily commodity market arrivals, price bands (Min, Max), and Modal trading rates (₹/Quintal and ₹/20kg Man) across 5 major APMC market yards in Vadodara District: **Sayajigunj, Padra, Karjan, Dabhoi, and Savli**. Features 24-hour price trend indicators and admin market rate overrides.

📥 WHERE DATA COMES FROM

- **Official Mandi Portals:** National Agriculture Market (e-NAM) & Data.gov.in Agmarknet daily price bulletins for Gujarat mandis.
- **Regional Trade Slips:** Daily trading auction averages collected from APMC Secretaries across Vadodara district.
- **Administrative Overrides:** Real-time price updates submitted by district admins via `adminRoutes.js`.
- **Pre-Ingested Seed Data:** `backend/src/utils/mandiData.js` benchmark rate database.

💾 WHERE DATA IS STORED

- **MongoDB Atlas Collection:** `mandiprices`
- **Model File:** `backend/src/models/MandiPrice.js`
- **Stored Schema Fields:** `market` (Sayajigunj, Padra, etc.), `commodity` (Cotton, Castor, Pigeon Pea, etc.), `variety`, `minPrice`, `maxPrice`, `modalPrice`, `arrivalsTonnes`, `tradeDate`, `trend` ('up' | 'down' | 'stable').
- **Seeder Script:** `backend/src/utils/seedMandiData.js` seeds fresh daily commodity rates.

⚙️ BACKEND PROCESSING FILES

- `backend/src/routes/mandiRoutes.js`: Exposes `GET /api/mandi`, `GET /api/mandi/trends`, `POST /api/mandi/update`.
- `backend/src/controllers/mandiController.js`: Filters rates by mandi yard, calculates modal average across district, computes 24-hour percentage delta.
- `backend/src/utils/seedMandiData.js`: Automatic MongoDB seeder invoked on server startup if collections are empty.

🖥️ FRONTEND UI PRESENTATION

- `frontend/src/pages/MandiPrices.jsx`: Live ticker banner, market yard tab switcher, search filter, rate cards with high-contrast badge indicators.
- `frontend/src/services/mandiService.js`: Fetches live rates from `/api/mandi` with automatic offline fallback to local benchmark constants.
- Bilingual rate display: English (₹/Quintal) & Gujarati (રૂ./મણ - ૨૦ કિલો).

END-TO-END DATA FLOW PIPELINE

1. Farmer visits `MandiPrices.jsx` and clicks "Padra APMC".
2. `mandiService.js` triggers `GET /api/mandi?market=Padra`.
3. `mandiController.js` queries MongoDB Atlas `mandiprices` collection for Padra yard records.
4. Modal rates (e.g. Cotton ₹7,450/Qtl, Tomato ₹1,850/Qtl) and arrival volumes are returned as JSON.
5. UI displays high-contrast cards with green "+₹120 (1.6%)" upward trend badges and Gujarati commodity names (કપાસ, ટામેટા).

Module 4: Smart Root-Zone Irrigation Timer & MGVCL Power Roster

Precision Water Conservation & DISCOM Scheduling

Calculates exact crop water volume requirements (Liters) and submersible pump runtimes (Minutes) based on crop evapotranspiration coefficients (K_c), soil texture moisture retention, and acreage. Directly synchronizes with the **MGVCL (Madhya Gujarat Vij Company Ltd) 8-hour agricultural 3-phase power shift roster** with live audio shutdown chimes.

📥 WHERE DATA COMES FROM

- **Agronomic Crop Coefficients (K_c):** FAO-56 standard coefficients calibrated for Gujarat crops (Cotton 0.85, Banana 1.10, Pigeon Pea 0.70, Tomato 0.95).
- **Soil Moisture Capacity:** Goradu Sandy Loam (12% moisture retention) vs Black Cotton Soil (28% retention).
- **MGVCL DISCOM Roster:** Vadodara rural 3-phase power shift schedules (Day shift 08:00 - 16:00, Night shift 22:00 - 06:00).
- **Farmer Inputs:** Field acreage (0.5 to 25 Acres) and irrigation method (Drip, Sprinkler, Flood).

💾 WHERE DATA IS STORED

- **MongoDB Atlas Collection:** `irrigationlogs`
- **Model File:** `backend/src/models/IrrigationLog.js`
- **Stored Schema Fields:** `crop`, `soilType`, `acreage`, `waterRequiredLiters`, `recommendedMinutes`, `method`, `powerShift`, `completedAt`, `litersSaved`.
- **Client State:** Live ticking countdown timer state persisted in browser memory.

⚙️ BACKEND PROCESSING FILES

- `backend/src/routes/irrigationRoutes.js`: `POST /api/irrigation/calculate` & `POST /api/irrigation/log`.
- Water formula: `Volume = Acreage × 4046.86 m² × ETo (4.5mm) × Kc × (1 - SoilFactor) × MethodEff`.
- Runtime formula: `Minutes = Volume (Liters) / (Pump HP × 250 L/min × 60)`.

🖥️ FRONTEND UI PRESENTATION

- `frontend/src/pages/IrrigationTimer.jsx`: Interactive soil & crop selectors, live circular countdown clock, pump start/pause controls, MGVCL 3-phase power roster badge.
- Web Audio API synthesizer sounding an alarm chime upon timer completion to notify farmer to shut off submersible pump.
- Cumulative water savings counter in liters.

END-TO-END DATA FLOW PIPELINE

1. Farmer inputs: 3.5 Acres, Cotton crop, Goradu Sandy Loam soil on `IrrigationTimer.jsx`.
2. Client calculates requirement: 28,500 Liters, 48 Minutes pump runtime on a 5HP borewell motor.
3. Farmer hits "Start Irrigation"; UI triggers interactive countdown clock with MGVCL power shift check.
4. When timer reaches 00:00, pump shutdown chime rings; session is logged via `POST /api/irrigation/log` to MongoDB Atlas.

Module 5: Agro-Weather Observatory & IMD Early Warning System

Microclimate Monitoring & Emergency Alerts

Delivers high-precision agro-meteorological monitoring for Vadodara District ($22.3072^\circ ext{N}, 73.1812^\circ ext{E}$) including ambient temperature, relative humidity, wind speed, solar radiation, heatwave risk indexing, and instant broadcasting of **IMD (India Meteorological Department) Ahmedabad** advisory warnings.

📥 WHERE DATA COMES FROM

- **Open-Meteo Weather API:** Live hourly and 7-day weather forecasts for Vadodara coordinates (100% keyless open API).
- **IMD Ahmedabad Radar Bulletins:** Official meteorological warnings regarding heatwaves (લૂ), unseasonal rainfall (માવઠું), and high-wind thunderstorms.
- **District Admin Emergency Alerts:** Bulletins dispatched in real time via the Operations Console.

💾 WHERE DATA IS STORED

- **MongoDB Atlas Collection:** `weatheralerts`
- **Model File:** `backend/src/models/WeatherAlert.js`
- **Stored Schema Fields:** `severity` ('info' | 'warning' | 'severe'), `title`, `titleGu`, `message`, `messageGu`, `source`, `validUntil`, `isActive`.
- **Fallback:** Seeded IMD heatwave notice loaded if Atlas connection is unavailable.

⚙️ BACKEND PROCESSING FILES

- `backend/src/routes/weatherRoutes.js`: `GET /api/weather/current` & `GET /api/weather/alerts`.
- `backend/src/models/WeatherAlert.js`: Defines active status and expiration index.
- Calculates Heat Index (apparent temperature) and foliar spraying advisories based on wind speed (<15 km/h for pesticide application).

🖥️ FRONTEND UI PRESENTATION

- `frontend/src/pages/Weather.jsx`: Real-time atmospheric dial (Temperature, Humidity, Wind, UV Index, Heatwave risk gauge).
- `frontend/src/components/AlertCard.jsx`: Persistent top banner warning farmers to avoid afternoon watering during peak heat.
- Bilingual rendering with Gujarati agro-weather vocabulary (ગરમીની ચેતવણી, ભેજ, પવનની ઝડપ).

END-TO-END DATA FLOW PIPELINE

1. `Weather.jsx` mounts and calls Open-Meteo API for Vadodara coordinates.
2. Simultaneously queries `/api/weather/alerts` to retrieve active emergency bulletins.
3. Backend checks MongoDB Atlas `weatheralerts` for non-expired alerts (`validUntil > now`).
4. Frontend renders 7-day hourly temperature curve and displays IMD alert banner at the top of the Home and Weather pages.

Module 6: Krishi Tools & Equipment Marketplace (i-Khedut Subsidies)

Farm Machinery Sharing & State Subsidies

Agricultural equipment rental sharing portal connecting smallholder farmers in Vadodara with **Custom Hiring Centers (CHCs)** across all 7 talukas. Features automated calculation of **Gujarat State i-Khedut Portal Subsidies (40% to 60%)** for spray drones, rotavators, laser levelers, and solar pumps with booking receipts.

📥 WHERE DATA COMES FROM

- **CHC Equipment Inventory:** Machinery specs, daily rental rates, and hub locations across Padra, Karjan, Dabhoi, Savli, Waghodia, Shinor, and Vadodara Rural.
- **State i-Khedut Subsidy Slabs:** Government subsidy slabs (40% standard farmer subsidy, 50–60% small/marginal farmer subsidy).
- **Farmer Booking Form:** Rental duration (days), required start date, landholding size, and contact details.

💾 WHERE DATA IS STORED

- **MongoDB Atlas Collections:** `equipments` and `equipmentbookings`
- **Model Files:** `backend/src/models/Equipment.js` & `backend/src/models/EquipmentBooking.js`
- **Stored Schema Fields:** `name`, `category`, `dailyRentalRate`, `subsidyPercentage`, `location`; Booking: `farmerId`, `grossRent`, `subsidyApplied`, `netPayable`, `bookingRef`, `status`.
- **Auto-Seeder:** `equipmentRoutes.js` auto-seeds 6 machinery models on first launch.

⚙️ BACKEND PROCESSING FILES

- `backend/src/routes/equipmentRoutes.js`: `GET /api/equipment`, `POST /api/equipment/book`, `GET /api/equipment/bookings/:userId`.
- Calculates gross cost, deducts i-Khedut subsidy amount, generates unique booking reference (e.g. `HRT-EQ-2026-XXXX`).
- Updates equipment availability status upon confirmed booking.

🖥️ FRONTEND UI PRESENTATION

- `frontend/src/pages/Marketplace.jsx`: 6 machinery catalog cards (Drone, Rotavator, Laser Leveler, Solar Pump, Harvester, Drip Kit).
- Interactive booking modal with live i-Khedut subsidy breakdown (Gross Rent - 50% State Subsidy = Net Farmer Cost).
- `frontend/src/services/equipmentService.js`: REST client dispatching booking requests to backend.

END-TO-END DATA FLOW PIPELINE

1. Farmer selects "Precision Agro Drone Sprayer" (₹1,800/day) on `Marketplace.jsx`.
2. Modal computes: 3 days = ₹5,400 gross. Applied i-Khedut 50% subsidy = ₹2,700 savings; Net farmer payable = ₹2,700.
3. Farmer clicks "Confirm Booking"; `equipmentService.js` posts booking payload to `/api/equipment/book`.
4. Backend saves record in MongoDB Atlas `equipmentbookings` and returns booking reference `HRT-EQ-8492`.
5. Frontend displays digital confirmation receipt with CHC dispatch contact details in Padra.

Module 7: District Admin Operations Console & AI Telemetry Hub

Operations Telemetry & Centralized Gating

Mission-control administrative console accessible strictly by authenticated **District Operations Administrators**. Provides real-time MongoDB Atlas shard latency ping telemetry, Node.js server memory metrics, Dual-Engine AI token consumption ledger (Cloud Gemini vs Offline TF.js), APMC price overrides, and emergency advisory broadcast.

📥 WHERE DATA COMES FROM

- **MongoDB Atlas Ping Telemetry:** Live execution of `mongoose.connection.db.admin().ping()` measuring cloud latency in milliseconds.
- **Node.js Server Heap Telemetry:** `process.memoryUsage()` capturing heapUsed, heapTotal, and RSS.
- **Dual-Engine AI Consumption Ledger:** Running counter tracking tokens consumed by Google Gemini Vision versus zero-cost offline TensorFlow.js edge inferences.
- **Admin Command Inputs:** Emergency alert broadcast text & APMC mandi rate adjustments.

💾 WHERE DATA IS STORED

- **Backend Memory State:** Live telemetry ring buffers maintained in `adminRoutes.js` for fast chart streaming.
- **MongoDB Collections:** Emergency alerts persisted directly into `weatheralerts`; mandi adjustments persisted to `mandiprices`.
- **Role Authentication:** Protected by JWT role verification (`role === 'admin'`).

⚙️ BACKEND PROCESSING FILES

- `backend/src/routes/adminRoutes.js`: Exposes `GET /api/admin/telemetry`, `POST /api/admin/alerts/broadcast`, `POST /api/admin/mandi/override`.
- Calculates financial savings in USD and INR achieved by offloading diagnostics to edge TensorFlow.js models.
- Validates administrator permissions before committing state modifications.

🖥️ FRONTEND UI PRESENTATION

- `frontend/src/pages/AdminPanel.jsx`: High-contrast dark green dashboard (`#1b4332`), live Atlas latency ping card (e.g. 14ms), heap memory usage bars, AI inference ledger comparison, emergency alert form.
- `frontend/src/services/adminService.js`: Polls telemetry endpoint and dispatches broadcast commands.
- Institutional gating: Desk button in navbar dynamically transforms into "Admin Console" (એડમિન કન્સોલ) only when logged in as admin.

END-TO-END DATA FLOW PIPELINE

1. Administrator signs in with `admin` role and navigates to `AdminPanel.jsx`.
2. `adminService.js` queries `GET /api/admin/telemetry`.
3. Backend executes Mongoose admin ping (e.g. 16ms), reads server memory (112MB Heap), and tallies token ledger.
4. Admin inputs a heatwave warning and clicks "Broadcast IMD Advisory"; backend writes alert to MongoDB `weatheralerts`.
5. All active farmer clients immediately receive and display the new advisory banner.

Module 8: Multi-Role Auth, User Desks & Agrarian Query System

Identity Management & Extension Officer Workspace

Provides secure, role-based authentication and operational desks for **Farmers**, **Agricultural Officers**, and **Administrators**. Farmers can register directly via the signup page, manage farm profiles, and submit agronomic queries to district officers who review and provide official answers.

📥 WHERE DATA COMES FROM

- **Signup Form:** 3-way Role Selector (👨‍🌾 Farmer, 🛡️ Admin, 🌾 Officer) with role-specific credential inputs.
- **1-Click Live Demo Switcher:** Instant login with authentic personas (Ramesh Patel - Farmer, Dr. Hitesh Trivedi - Officer, District Admin).
- **Farmer Query Input:** Crop inquiry subject, question body, and affected field location.

💾 WHERE DATA IS STORED

- **MongoDB Atlas Collections:** `users` and `queries`
- **Model Files:** `backend/src/models/User.js` & `backend/src/models/Query.js`
- **User Schema:** `name`, `email`, `password` (bcrypt salt=10), `role` enum ['farmer', 'officer', 'admin'], `district`, `taluka`, `farmSizeAcres`, `primaryCrops`, `department`, `designation`.
- **Query Schema:** `farmerId`, `farmerName`, `subject`, `question`, `officerResponse`, `status` ('pending' | 'resolved').

⚙️ BACKEND PROCESSING FILES

- `backend/src/routes/authRoutes.js`: `POST /api/auth/register`, `/login`, `/demo-login`, `GET /me`.
- `backend/src/routes/queryRoutes.js`: `POST /api/queries`, `GET /api/queries`, `PUT /api/queries/:id/respond`.
- `backend/src/middleware/authMiddleware.js`: Decodes JWT Bearer token and attaches user object to Express request.
- `backend/src/utils/generateToken.js`: Signs JWT tokens containing user ID and role with 30-day expiration.

🖥️ FRONTEND UI PRESENTATION

- `frontend/src/pages/LoginPage.jsx`: Dual-tab (Sign In / Register) with 3-way role buttons and dynamic admin credential fields.
- `frontend/src/components/AuthModal.jsx`: 1-Click demo switcher modal.
- `frontend/src/pages/UserProfile.jsx`: Dynamic desk switching based on role:
  - **Farmer Desk:** Scan history, acreage, crop list, query submission box.
  - **Officer Desk:** Pending farmer queries list, official reply editor, resolution badge.

END-TO-END DATA FLOW PIPELINE

1. Farmer registers or logs in; JWT token is saved to `localStorage('haritayan_auth_token')`.
2. Farmer submits query: "Whitefly infestation on Bt cotton in Karjan" on `UserProfile.jsx`.
3. `queryService.js` posts to `/api/queries`; stored in MongoDB Atlas `queries` as `status: 'pending'`.
4. Agricultural Officer Dr. Hitesh Trivedi logs in; his Officer Desk loads pending queries list.
5. Officer submits response: "Apply Neem Seed Kernel Extract (NSKE 5%)"; query is marked `status: 'resolved'` and farmer's desk updates immediately.

Module 9: HaritAyan Sahayak Bilingual Rule-Based Chatbot

Bilingual Conversational Agronomic Assistant

Offline-resilient conversational rule-based AI assistant available globally across every page of HaritAyan. Understands regional farmer vernacular in English and Gujarati, answering common agronomic inquiries, recommending organic pest solutions, and providing one-click deep navigation links across all platform modules.

📥 WHERE DATA COMES FROM

- **Farmer Chat Input:** Natural language queries entered via keyboard or quick prompt recommendation chips.
- **Agronomic Rule Base:** Curated decision tree covering crop diseases, weather heatwave advisories, APMC mandi trading hours, irrigation calculations, and i-Khedut subsidies.
- **Gujarati Botanical Lexicon:** Bilingual mapping for crops (કપાસ, દિવેલા, તુવેર), diseases (પાનના ટપકા, સુકારો), and mandis (પાદરા, કરજણ).

💾 WHERE DATA IS STORED

- **Client Memory State:** Active dialogue history array maintained in React component state.
- **Local Session Cache:** Retains active conversation session across page navigations in `sessionStorage`.
- **Rule Knowledge Base:** Bundled directly inside frontend client bundle for zero-latency, 100% offline reliability.

⚙️ PROCESSING ARCHITECTURE

- `frontend/src/components/Chatbot.jsx`: Implements regex and keyword token matching against agricultural query patterns.
- Computes intent score (Disease diagnosis, Mandi rates, Irrigation timer, Weather advisory, Equipment rental, User help).
- Synthesizes bilingual answer and attaches actionable page navigation buttons.

🖥️ FRONTEND UI PRESENTATION

- Floating trigger button with glowing leaf badge at bottom-right of viewport.
- Expandable chat window with animated message bubbles, avatar indicators, quick query suggestions ("Show cotton remedies", "કપાસના રોગો").
- Action buttons that programmatically route the user directly to the relevant tool (e.g. "Open Disease Scanner →", "View Mandi Rates →").

END-TO-END DATA FLOW PIPELINE

1. Farmer taps floating chatbot button and types: "What is cotton price in Padra APMC?".
2. Rule engine matches tokens `['cotton', 'price', 'padra']` to Mandi Intent.
3. Chatbot crafts instant response with today's modal price band and appends a "Navigate to APMC Mandi" action pill.
4. Farmer clicks action button; application instantly navigates to `MandiPrices.jsx` with Padra tab pre-selected.

3. MongoDB Atlas Schema & Collections Reference

Collection	Mongoose Model File	Primary Key & Essential Fields	Purpose & Associated Workflows
`users`	`models/User.js`	`_id`, `email` (unique), `password` (bcrypt hash), `role` ('farmer'|'officer'|'admin'), `district`, `taluka`, `farmSizeAcres`, `primaryCrops`	Authentication, user role gating, farmer profiles, officer designations, administrator privileges.
`scans`	`models/Scan.js`	`_id`, `userId`, `crop`, `disease`, `confidence`, `severity`, `symptoms[]`, `organicRemedies[]`, `imageUrl`, `inferenceEngine`	Foliar diagnostic history, remedial recommendations, ML training feedback loop, farmer scan records.
`mandiprices`	`models/MandiPrice.js`	`_id`, `market`, `commodity`, `minPrice`, `maxPrice`, `modalPrice`, `arrivalsTonnes`, `tradeDate`, `trend`	Daily APMC commodity price ticker across 5 district yards, historical price charts, admin overrides.
`equipments`	`models/Equipment.js`	`_id`, `name`, `category`, `dailyRentalRate`, `subsidyPercentage`, `location`, `specs`	Farm machinery rental catalog across Vadodara CHCs with state i-Khedut subsidy slabs.
`equipmentbookings`	`models/EquipmentBooking.js`	`_id`, `farmerId`, `equipmentId`, `startDate`, `durationDays`, `grossRent`, `subsidyApplied`, `netPayable`, `status`	Farmer machinery rental bookings, invoice generation, CHC equipment dispatch tracking.
`queries`	`models/Query.js`	`_id`, `farmerId`, `farmerName`, `subject`, `question`, `officerResponse`, `status` ('pending'|'resolved')	Farmer extension advisory portal, officer resolution queue, two-way agronomic consultation.
`irrigationlogs`	`models/IrrigationLog.js`	`_id`, `crop`, `soilType`, `acreage`, `waterRequiredLiters`, `recommendedMinutes`, `completedAt`, `litersSaved`	Water conservation audit logging, irrigation schedule optimization, DISCOM power roster records.
`weatheralerts`	`models/WeatherAlert.js`	`_id`, `severity`, `title`, `titleGu`, `message`, `messageGu`, `source`, `validUntil`, `isActive`	IMD Ahmedabad weather bulletins, heatwave alerts, emergency notices broadcast to all platform users.

4. Complete REST API Route Specification

Method	API Endpoint	Auth / Role	Controller Handler	Action & Output
`POST`	`/api/auth/register`	Public	`authController.registerFarmer`	Creates Farmer, Admin, or Officer account; returns JWT & safe user object.
`POST`	`/api/auth/login`	Public	`authController.loginUser`	Authenticates credentials with bcrypt; issues 30-day JWT token.
`POST`	`/api/auth/demo-login`	Public	`authController.demoLogin`	Instant 1-click persona switcher (Ramesh, Dr. Hitesh, Admin).
`GET`	`/api/auth/me`	Private	`authController.getCurrentUser`	Fetches logged-in user profile, crops, and taluka information.
`POST`	`/api/scan`	Public / Token	`scanController.analyzeLeafImage`	Multer uploads leaf photo; Gemini 1.5 Pro analyzes and returns JSON remedies.
`GET`	`/api/scan/history`	Private	`scanController.getUserScanHistory`	Retrieves farmer's chronological diagnostic scans from MongoDB.
`GET`	`/api/mandi`	Public	`mandiController.getMandiPrices`	Returns daily commodity rates filtered by mandi yard & date.
`POST`	`/api/mandi/update`	Admin	`mandiController.updateMandiPrice`	Allows administrators to override daily commodity prices in real time.
`GET`	`/api/weather/current`	Public	`weatherController.getCurrentWeather`	Proxies Open-Meteo ambient observations for Vadodara coordinates.
`GET`	`/api/weather/alerts`	Public	`weatherController.getActiveAlerts`	Retrieves non-expired IMD weather and heatwave advisories.
`GET`	`/api/heat/wards`	Public	`heatController.getWardHeatData`	Returns baseline thermal readings and NDVI for all 8 Vadodara wards.
`POST`	`/api/heat/simulate`	Public	`heatController.simulateCanopyCooling`	Computes microclimatic cooling delta based on tree species and count.
`POST`	`/api/irrigation/calculate`	Public	`irrigationController.calculate`	Calculates root-zone moisture liters and pump minutes.
`POST`	`/api/irrigation/log`	Private	`irrigationController.logSession`	Logs completed watering runtime and cumulative liters saved.
`GET`	`/api/equipment`	Public	`equipmentController.getCatalog`	Returns 6 machinery listings with i-Khedut subsidy percentages.
`POST`	`/api/equipment/book`	Private	`equipmentController.createBooking`	Applies 40-60% state subsidy; issues confirmed booking reference.
`GET`	`/api/admin/telemetry`	Admin	`adminController.getTelemetry`	Executes Atlas shard ping (ms), reads memory heap,

Method	API Endpoint	Auth / Role	Controller Handler	Action & Output
				and counts AI token ledger.
`POST`	`/api/admin/alerts/broadcast`	Admin	`adminController.broadcastAlert`	Dispatches emergency weather advisory directly to all client banners.
`POST`	`/api/queries`	Farmer	`queryController.submitQuery`	Submits agronomic question to agricultural extension officers.
`PUT`	`/api/queries/:id/respond`	Officer	`queryController.respondToQuery`	Agri Officer submits official diagnosis and treatment to resolve query.